

Software Requirements Specification (SRS)

Smart Inventory Management System

Version: 1.0

Project Type: Full Stack Web Application

Backend: ASP.NET Core Web API

Frontend: Angular

Database: SQL Server

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the Smart Inventory Management System. The system is designed to help manage products, warehouses, and inventory movements in a simple and secure way.

1.2 Scope

The system provides:

- User authentication with JWT
- Role-based authorization
- Product management
- Warehouse management
- Inventory tracking
- Stock IN / OUT transactions
- Inventory history
- Pagination for large data sets

The project is intended to be a clean and practical full-stack application that demonstrates modern backend and frontend development practices.

1.3 Definitions

- **Product:** A stock item managed by the system.
 - **Warehouse:** A physical storage location.
 - **Inventory:** The current quantity of a product inside a warehouse.
 - **Inventory Transaction:** A record of a stock movement, such as stock IN or stock OUT.
 - **JWT:** JSON Web Token used for secure authentication.
 - **Admin:** A user with full access to manage core data.
 - **Employee:** A user with limited access based on authorization rules.
-

2. Overall Description

2.1 Product Perspective

The system is a web-based inventory application with:

- ASP.NET Core Web API backend
- Angular frontend
- SQL Server database
- JWT-based authentication
- Role-based access control

2.2 User Classes

Admin

- Can manage products
- Can manage warehouses
- Can manage inventory transactions
- Can access all data

Employee

- Can view products
- Can view warehouses
- Can create inventory transactions
- Can view inventory history if allowed by permissions

2.3 Assumptions

- SQL Server is available.
- The user has a supported browser.

- The backend and frontend are deployed or run locally.
- An initial admin account is seeded.

2.4 Constraints

- The system must use JWT authentication.
 - The system must use role-based authorization.
 - The system must prevent negative stock.
 - The system must support the required APIs.
 - The frontend must use Angular.
 - The project must follow a clean and maintainable structure.
-

3. Functional Requirements

3.1 Authentication

FR-1 Login

The system shall allow users to log in using email and password.

FR-2 JWT Token Generation

After a successful login, the system shall return a JWT token.

FR-3 Role-Based Authorization

The system shall restrict actions based on user roles such as Admin and Employee.

FR-4 Logout

The system shall allow users to log out and remove their token from the client.

3.2 Product Management

FR-5 View Products

The system shall allow authenticated users to view a list of products.

FR-6 Get Product by Id

The system shall allow authenticated users to view product details by id.

FR-7 Create Product

The system shall allow Admin users to create a new product.

FR-8 Update Product

The system shall allow Admin users to update an existing product.

FR-9 Delete Product

The system shall allow Admin users to delete a product.

FR-10 SKU Validation

The system shall ensure that product SKU values are unique.

3.3 Warehouse Management

FR-11 View Warehouses

The system shall allow authenticated users to view warehouses.

FR-12 Create Warehouse

The system shall allow Admin users to create a warehouse.

FR-13 Update Warehouse

The system shall allow Admin users to update a warehouse.

FR-14 Delete Warehouse

The system shall allow Admin users to delete a warehouse.

FR-15 Warehouse Validation

The system shall prevent deleting a warehouse if it still has active inventory.

3.4 Inventory Management

FR-16 Stock IN Transaction

The system shall allow users to increase inventory quantity using a stock IN transaction.

FR-17 Stock OUT Transaction

The system shall allow users to decrease inventory quantity using a stock OUT transaction.

FR-18 Negative Stock Prevention

The system shall prevent inventory quantity from becoming negative.

FR-19 Inventory History

The system shall store and return a history of inventory transactions.

FR-20 Inventory Tracking

The system shall track current product quantity per warehouse.

3.5 Pagination

FR-21 Paginated Results

The system shall support pagination for large data sets where needed.

4. Non-Functional Requirements

4.1 Security

- The system shall use JWT authentication.
- The system shall protect endpoints using role-based authorization.
- The system shall store passwords securely using a safe authentication mechanism.
- The system shall use HTTPS in production.
- The system shall validate all incoming data.

4.2 Performance

- The system shall return common API responses in a reasonable time.
- The system shall support pagination to avoid loading huge lists at once.
- The system shall optimize inventory reads using current state storage.

4.3 Reliability

- The system shall use centralized exception handling.
- The system shall ensure inventory updates and transaction inserts are saved atomically.

4.4 Maintainability

- The system shall follow Clean Architecture principles.
- The system shall use DTOs to separate API contracts from database entities.
- The system shall use services and repositories to separate concerns.

4.5 Usability

- The system shall provide a simple and clear UI.
 - The system shall show loading and error states.
 - The system shall provide validation messages for user input.
-

5. Use Cases

5.1 Use Case: User Login

Actor: Admin or Employee

Description: The user enters credentials and receives a JWT token if valid.

Main Flow:

1. User opens login page.
2. User enters email and password.
3. System validates credentials.
4. System returns JWT token.
5. Client stores token.
6. User is redirected to dashboard.

Alternative Flow:

- If credentials are invalid, the system displays an error message.
-

5.2 Use Case: Create Product

Actor: Admin

Description: Admin creates a new product.

Main Flow:

1. Admin opens product form.
2. Admin enters product details.
3. System validates input.
4. System checks SKU uniqueness.
5. System saves the new product.
6. System returns success message.

Alternative Flow:

- If SKU already exists, the system returns validation error.
-

5.3 Use Case: Create Warehouse

Actor: Admin

Description: Admin creates a warehouse.

Main Flow:

1. Admin opens warehouse form.
 2. Admin enters warehouse details.
 3. System validates input.
 4. System saves the warehouse.
 5. System returns success message.
-

5.4 Use Case: Stock IN Transaction

Actor: Admin or Employee

Description: User adds stock to a warehouse.

Main Flow:

1. User opens inventory transaction form.
2. User selects product and warehouse.
3. User enters quantity.

4. System validates product and warehouse.
 5. System checks if inventory item exists.
 6. System increases quantity.
 7. System stores the transaction history.
 8. System returns success.
-

5.5 Use Case: Stock OUT Transaction

Actor: Admin or Employee

Description: User removes stock from a warehouse.

Main Flow:

1. User opens inventory transaction form.
2. User selects product and warehouse.
3. User enters quantity.
4. System checks available quantity.
5. System decreases quantity.
6. System stores the transaction history.
7. System returns success.

Alternative Flow:

- If quantity is insufficient, the system returns an error.
-

6. External Interface Requirements

6.1 User Interface

The frontend shall contain:

- Login page
- Dashboard
- Products list page
- Add/Edit product page
- Warehouses page
- Inventory transaction page
- Inventory history page

6.2 API Interface

Required APIs:

Authentication

- `POST /api/auth/login`

Products

- `GET /api/products`
- `GET /api/products/{id}`
- `POST /api/products`
- `PUT /api/products/{id}`
- `DELETE /api/products/{id}`

Warehouses

- `GET /api/warehouses`
- `POST /api/warehouses`

Inventory

- `POST /api/inventory/in`
- `POST /api/inventory/out`
- `GET /api/inventory/history`

6.3 Database Interface

The system shall use SQL Server and Entity Framework Core for persistence.

7. Data Requirements

7.1 Main Entities

- User
- Role
- Product
- Warehouse
- InventoryItem

- InventoryTransaction

7.2 Entity Relationships

- One Role has many Users
- One Product can appear in many InventoryItems
- One Warehouse can contain many InventoryItems
- One Product can have many InventoryTransactions
- One Warehouse can have many InventoryTransactions
- One User can create many InventoryTransactions

7.3 Data Rules

- Product SKU must be unique.
- Inventory quantity must never be negative.
- Inventory transactions must be stored as history.
- Deleted products and warehouses should not break historical data.

8. Error Handling Requirements

- The system shall use global exception handling.
- The system shall return meaningful error responses.
- The system shall handle not found, validation, and business rule violations.
- The frontend shall display user-friendly error messages.

9. Validation Requirements

- Required fields must not be empty.
- Price must be greater than zero.
- Quantity must be greater than zero for transactions.
- Product SKU must be unique.
- Warehouse name must be unique.
- Stock OUT must not exceed available quantity.

10. Testing Requirements

10.1 Unit Tests

The system shall include unit tests for important business logic such as:

- Inventory transaction processing
- Stock validation
- Product validation
- Warehouse deletion rules

10.2 Integration Tests

Optional integration tests may be used to verify API flow and database behavior.

11. Deployment Notes

- Backend can run locally using ASP.NET Core.
- Frontend can run using Angular development server.
- Database migrations shall be applied before running the system.
- User secrets shall be used for local development configuration.

12. Future Enhancements

Possible future improvements:

- Search and filtering
- Docker support
- Dark mode
- Caching
- Reporting dashboard
- Export to PDF/Excel

13. Acceptance Criteria

The project will be considered complete if:

- Users can log in successfully.
- JWT authentication works.
- Role-based access works.
- Products can be managed.

- Warehouses can be managed.
 - Inventory transactions work correctly.
 - Negative stock is prevented.
 - Inventory history is available.
 - Pagination works.
 - Unit tests pass.
 - README, SRS, and database script are included.
 - The project is uploaded to GitHub successfully.
-

14. Conclusion

The Smart Inventory Management System is a simple but complete inventory solution built to demonstrate full-stack development skills, secure authentication, clean architecture, and proper system design.